Certified Trajectory Optimization on Time-Layered Graphs

Algorithm proposal: from trellis to certified trajectory

Zihang Huang · November 2, 2026 · draft

The proposal in brief

- **Goal.** For a contact-rich robot, return a feasible trajectory with cost U and a certified lower bound $L \le J^\star$, with a small gap $U - L$, fast enough to run online.
- **Model.** A time-layered graph (trellis): one layer per time step, one vertex per contact mode. Each edge is one time step in one mode. Its true set is nonconvex. The solver uses a convex outer model of it.
- **Solver.** PDHG on the fractional flow relaxation. Edges update in parallel, prices live on vertices. Most edge solves are skipped exactly. An optional tree pass adds long-range corrections.
- **Lower bound.** From prices alone: a shortest path over per-edge priced minima. It is valid for *any* prices. It is free every round, and tighter when edges are evaluated on their true sets.
- **Upper bound.** Pick mode sequences from the flows and the bound's own paths. Run a local solver along each with the contact schedule fixed.
- **Loop.** Remove edges that provably cannot help, split modes where the gap sits, and repeat until $U - L \le \eta$.
- **Status.** Every bound is valid by proof. Speed and tightness are empirical.

1 Problem and setting

In short. Find a good trajectory and prove how good it is.

- **System.** State $x = (q, \dot{q})$ of dimension D, controls u, contact forces λ, manipulator dynamics $M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = Bu + J(q)^\top \lambda$.
- **Contact modes.** M modes (which contacts are active, sticking or sliding). Each mode has its own constraints and dynamics.
- **Horizon.** N time steps of length Δt, from start x_0 to goal x_g.
- **Output.** A feasible trajectory with cost U, and a number L with $L \le J^\star \le U$.
- **Tolerances.** State mismatch at vertices up to δ; target gap η.

2 Formulation: the time-layered trellis

In short. A shortest-path problem over modes, with a continuous state carried along each path.

- **Vertices.** (k, m): time step k, mode m. Plus a start and a goal vertex.
- **Edges.** $e : (k, m) \to (k+1, m')$ for each allowed transition: about NM^2 edges.
- **Edge set.** X_e holds (x, u, λ, x') for one step in mode m, with the switch to m': dynamics, contact and friction constraints, guards, limits. Cost ℓ_e. X_e is *nonconvex* (through $M(q)$, $J(q)$, C, G, rotations).
- **Convex outer model.** $\hat{X}_e \supseteq X_e$, with cost $\hat{\ell}_e \le \ell_e$ (Section 3).
- **Fractional relaxation.** Each edge carries a flow $y_e \in [0, 1]$ and weighted variables $z_e = y_e \cdot$

(x, u, λ, x'), constrained to the perspective of $\hat{X}_e$. The cost is the perspective of $\hat{\ell}_e$. Flow is conserved at every vertex, and states are conserved up to δ.

- **Validity.** Any true trajectory is feasible for the relaxation at no greater cost, so the relaxation value is at most $J^\star$.
- **Two sources of gap.** *Mixing*: flows from different paths average at vertices. *Convexification*: $\hat{X}_e$ is larger than X_e.

3 Convex outer models for each edge

In short. Exploit the fact that the dynamics are affine in accelerations, torques and forces. Bound the configuration-dependent terms over each mode's region.

- **Where the nonconvexity is.** For fixed $(q, \dot{q})$ the dynamics are affine in $(\ddot{q}, u, \lambda)$. All of the nonconvexity enters through $M(q)$, $J(q)$, $C(q, \dot{q})$, $G(q)$.
- **Option A: Taylor model with remainder.** Linearize around the center of the mode's region. Bound the remainder with second-derivative constants computed from the robot model (link inertias, joint ranges, velocity limits). For revolute joints these terms are trigonometric polynomials with bounded derivatives. The result is a polyhedral or conic set containing X_e. Its error scales like (region radius)^2.
- **Option B: per-edge QCQP relaxation.** In Newton–Euler form the dynamics are bilinear in per-link variables (rotations, twists, wrenches), a linear number of them. A degree-2 (Shor) relaxation per edge is then one semidefinite block per edge. That is heavy but local and parallel. No hierarchy over the whole trajectory is needed, because the trellis already separates time steps.
- **Regions as modes.** A mode can be split into subregions. Each subregion becomes its own mode, with a tighter outer model. This is spatial branching expressed inside the trellis.
- **Computed offline.** For a target per-edge error, Option A gives the region size needed before any solve is run.

4 Solver core: PDHG on the trellis

In short. Parallel edge steps, price updates on vertices, exact skipping of unnecessary edge solves, and an optional global correction.

One PDHG round.

1. **Edges, in parallel.** Each edge minimizes its cost plus price charges plus a proximal term, over its homogenized outer set.
2. **Prices, in parallel.** Each vertex price moves by its (extrapolated) mismatch, adjusted for δ.

Diagonal step sizes from vertex degrees make the step rule local. Restarts help on sharp (polyhedral) problems. On convex outer models, PDHG converges to the relaxation optimum.

Exact edge skipping. Each edge keeps one dual vector μ_e.

- **Zero test.** An edge's step is exactly zero if and only if its cheapest priced unit-flow transition costs at least zero. A weak-duality bound with μ_e certifies this with one pass over the edge's data. This holds for any convex edge.
- **Unchanged face.** For polyhedral edges, a step on last round's active face is a fixed linear map, verified exactly by a KKT check. The factorization is shared by all time steps with the same mode

pair and face. For conic edges this becomes a warm-started Newton step on the face. That is exact only to a tolerance, and PDHG still converges if the errors shrink fast enough.

- **Effect.** Iterates are unchanged, so all PDHG guarantees carry over. Genuine solves track switching events, not rounds times edges. Under strict complementarity and nondegeneracy, they stop entirely after finitely many rounds.

Optional tree pass. Freeze active sets. Then the price operator is block tridiagonal by layer (block size $M(D+1)$), because edges only join adjacent layers. Eliminating layer by layer on a segment tree (cyclic reduction) solves the linearized system exactly in $2\log_2 N$ levels. Use it as an extra direction behind a safeguard (SuperMann or a hybrid proximal extragradient test). Convergence is kept. Once active sets settle, it acts like a Newton step. It cannot see through edges with zero flow, so it does not speed up early discovery.

Warm starts. When replanning online, start from the previous cycle's flows, prices, cached faces and stored duals.

5 Lower bound: the certificate

In short. Any prices give a valid bound. It is free to evaluate every round, and evaluating edges on their true sets makes it tighter.

For state prices λ, let $\pi_e(\lambda)$ be edge e's minimum of cost plus price charges over one unit of flow. Then

$$L(\lambda) = \mathrm{SP}\big[\pi_e(\lambda)\big] - \delta \textstyle\sum_v \|\lambda_v\|_* \;\le\; J^\star,$$

where SP is the shortest path over the trellis. This holds whether π_e is evaluated on $\hat{X}_e$ or on the true X_e, and it holds for any lower bound on π_e.

- **Every round, no solves.** Replace π_e by its dual bound from the stored μ_e. The cost is one pass over the edges plus a dynamic program, $O(NM^2)$.
- **Tighter, periodically.** Evaluate π_e on the true nonconvex X_e: global solution, a Taylor-model bound, a per-edge Shor bound, or a small branch-and-bound. Each edge's true priced minimum is at least its outer-model value. So at the outer model's optimal prices, this bound is at least the outer model's bound.
- **Lazy exact evaluation.** Evaluate only the edges on the current shortest path, update, and repeat. This ends with the exact bound. In tests it needed about one path's worth of edges.
- **Ceiling.** No choice of prices beats the relaxation with each edge replaced by its convex hull. Going beyond that requires splitting modes.
- **Soundness.** Numerical error in each edge's bound is signed and included.

6 Upper bound: extracting a trajectory

In short. The relaxation proposes mode sequences and starting points. A local solver makes them exactly feasible.

Candidate mode sequences (several, solved in parallel):

- heaviest paths in the flow's path decomposition;
- randomized rounding: sample paths with probability proportional to flow;
- the shortest path from the certificate (free each round), and k-shortest paths;

- greedy: from the start, follow the edge with the most flow.

Initialization. Along the chosen path, use z_e/y_e as the initial guess, or the priced minimizers when the path came from the certificate.

Local solve. Run a local trajectory optimizer (e.g. PHLAME, AGHF, DDP) with the contact schedule fixed. The best feasible result gives U.

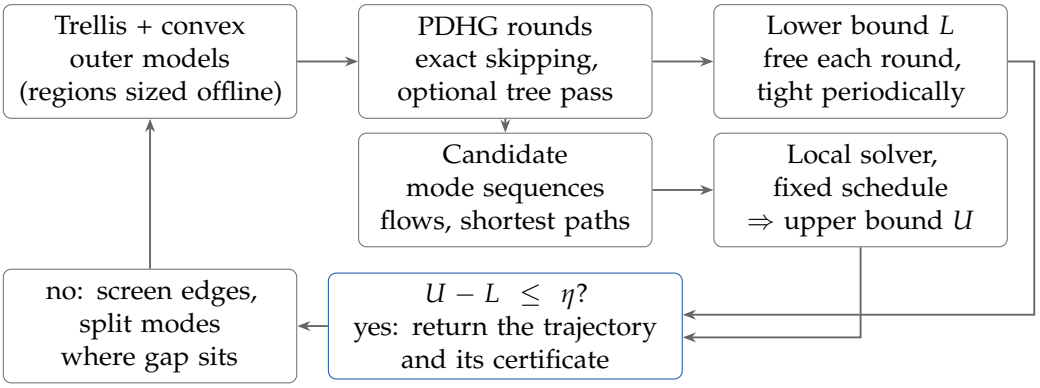
A direct read-off without a local solve works only when the flow is on a single path, the states satisfy the true dynamics, and they agree at vertices. That is rare when flows mix.

7 Screening and refinement

In short. Remove what provably cannot help. Refine where the gap actually is.

- **Safe screening.** An edge's through-cost is the best prefix, plus its priced bound, plus the best suffix. If it exceeds U, remove the edge. The optimal trajectory survives, the removal is permanent, and the relaxation can only tighten.
- **Mode refinement.** Split a mode's region where the outer model is loose: where an edge's true priced minimum differs most from its outer-model value, or where flows mix. Splitting reduces the convexification error, quadratically in region size for Option A. Its effect on mixing is empirical.
- **Optional factoring.** Splitting each layer into M dynamics edges and M^2 switch edges cuts heavy solves by a factor of M. It is exact only for a formulation without per-copy pre-state constraints, and it gave a 0.5–30% looser bound in tests. Use it only if solve count dominates.

8 The full loop



Algorithm.

1. **Offline:** build the trellis; size mode regions for the target per-edge error; precompute outer models and shared factorizations.
2. **Initialize:** flows, prices, caches and duals from the previous cycle, or zero on a cold start.
3. **Repeat** PDHG rounds. In each round: (a) zero test, else cached face, else solve, for every edge in parallel; (b) price update; (c) optionally, a tree pass behind a safeguard; (d) free lower bound L.
4. **Every few rounds:** tighten L with true-set edge bounds, lazily along shortest paths; extract candidate mode sequences; run local solves in parallel; update U.
5. **Stop** when $U - L \leq \eta$, or when the time budget ends; return the best trajectory and the current certificate.
6. **Otherwise,** when progress stalls: screen edges against U; split modes where the gap concentrates; continue from the current state.

9 What is guaranteed, and what is not

Claim	Status	Note
$L \leq J^\star$ at every round	Proven	Any prices; any valid per-edge bounds; signed numerical error
U is a real trajectory's cost	By construction	Checked by the local solver's feasibility
PDHG converges on outer models	Proven	$O(1/k)$ averaged gap in the worst case
Rounds: $N \lesssim R \lesssim \|K\| R_x R_p / \eta$	Proven	$R_p \leq (U - L_0)/\delta$; upper end pessimistic
Edge skipping changes nothing	Proven	Polyhedral edges exactly; conic edges to tolerance
Eventually no edge solves	Proven	Needs strict complementarity and nondegeneracy
Tree pass: exact linear solve, safeguarded	Proven	Speed-up only once active sets settle
Screening is safe	Proven	Removal is permanent and only tightens
Gap size $U - L$	Empirical	Ceiling: per-edge convex hulls plus mixing
Rounds to reach η	Empirical	No general bound beyond the bracket above
Solve savings at real scale	Empirical	Edge size changes how often faces switch

10 Cost model

- **Per round:** one cheap pass over all $E \approx NM^2$ edges (a dot product or a small matrix–vector product each); genuine solves only at switching events; one price update; one $O(E)$ shortest path.
- **Periodic:** true-set edge bounds along shortest paths; local solves for a few candidates, in parallel.
- **Optional tree pass:** about $N(M(D+1))^3$ work in $\log_2 N$ levels, at most about one round of edge solves at full width.
- **Wall-clock:** $T \approx \sum_{\text{rounds}} \left(\text{cheap pass} + \lceil \text{solves}/P \rceil \, C\right) + T_{\text{bounds}} + T_{\text{local}}$, with P the parallel capacity and C one edge solve.
- **Measured on a small polyhedral test** ($D = 2$, $M = 4$–8, $N = 20$): a 10,000-round run needed as many solves as 25–46 full rounds. With conic edges, the exact zero test alone cut solves 2–7×. Savings grow with run length. Against warm-started solvers they would be smaller.

11 Open questions (empirical)

Q1. Gap. How large are mixing and convexification at Go2 and hand scale, and how fast does mode refinement close them?

Q2. True-set edge bounds. Do Taylor models or per-edge Shor stay tight at $D \approx 40$, and at what cost?

Q3. Rounds. How many rounds reach the target gap, cold and warm-started?

Q4. Edge savings at scale. How often do faces switch for large edges, and how well does Newton-on-face work for conic edges?

Q5. Online. How large is the per-cycle gain from warm starts, and what does the GPU implementation cost?

12 Low-rank segment-tree correction

In short. Use a cheap model to coordinate changes across the whole horizon, then check the proposed change against the actual edge problems. This is a cheaper alternative to Section 4's full-width tree.

12.1 What the tree is trying to predict

An edge update responds to changes in its endpoint prices. The full tree retains a complete linear model of these responses and combines neighboring intervals until it can propose a correction for the whole trajectory. Its boundary width is $w = M(D+1)$: all modes and state coordinates remain present. Dense matrix operations at that width can be expensive.

The low-rank version keeps a simple reference response everywhere and adds only a few correction directions. It approximates the *optimizer's response*, not the robot's feasible set. The original convex outer models, contact constraints, objective and final certificate remain the ones declared earlier.

12.2 Why use a simple reference motion?

The reference supplies a cheap description of how a change propagates through time. For example, a position–velocity model retains the fact that changing velocity now also changes position later. A segment can therefore coordinate a displacement or a trend across many steps, rather than treating each boundary independently.

This is not one prescribed trajectory for the robot to follow. The same small reference model is repeated across coordinates and mode pairs, so every coordinate can change without storing a dense coupling between all of them. It supplies the basic temporal dependence; observed edge responses supply corrections for couplings that it misses.

The reference can be too simple. If the missing contact or joint couplings dominate the useful direction, its proposal will be poor. We do not assume that the reference is accurate: the actual progress check below decides whether its correction is usable. It can also propose changes through currently inactive edges where a local derivative is zero, although that does not guarantee successful activation.

12.3 What is retained at each segment

For each segment, retain a structured reference boundary matrix and at most r extra response directions. Schematically, the chosen positive correction model has the form

$$\widetilde{S}_I = S_I^{\mathrm{ref}} + U_I U_I^\top, \qquad \operatorname{rank} U_I \le r.$$

Here I is a time interval and the columns of U_I encode additional combinations of boundary changes. They can be built from differences between saved edge inputs and outputs. The rank cap limits the *proposal model*; it does not assert that the true mechanics have low rank.

When two neighboring segments merge, eliminate their shared boundary, retain at most r correction directions again, and save the information needed to recover interior changes on the downward pass. The cap applies at every level, including the root. The elimination factors and right-hand-side information must remain consistent with the compressed model. Ordinary averages of unrelated states, or truncating only a matrix and forgetting its right-hand side, are insufficient.

12.4 How it fits into one iteration

1. **Build or refresh the proposal model.** Use saved local responses where possible. Count any new edge solves, derivative evaluations or probing solves used to improve it.
2. **Pass upward, then downward.** Merge segment summaries in parallel at each level, solve the small root problem, then distribute the correction. Form an equation for a *change* about the current point, with the uncompressed chosen model's residual on the right. A zero residual must produce zero correction even after compression.
3. **Check the actual update.** Evaluate the required original edge updates, using exact skipping when its current-input checks pass. Apply the established progress safeguard from Section 4. If it fails, reject or damp the proposal according to that safeguard, or take the ordinary PDHG fallback. Each extra trial costs work.

The reference model's predicted cost is never substituted for the lower bound in Section 5. A tree step may cross several active regions, so the exact-reuse hit rates measured for ordinary PDHG need not carry over.

12.5 Cost, guarantees and the remaining question

For the shared reference with a fixed number of channels per coordinate and identical reference dynamics across all mode pairs of a complete trellis, internal tree construction costs $O(N[wr^2 + r^3])$, versus $O(Nw^3)$ for the dense exact-response tree. This excludes acquisition of local responses and aggregation over the E edges. A general mode graph can additionally require $O(NM^3)$ reference setup and $O(NM^2Dr)$ work to apply its factors to correction directions.

Both trees have $O(\log N)$ parallel levels. With serial arithmetic inside each merge, the capped tree's internal critical path is $O((wr^2 + r^3)\log N)$. Thus logarithmic tree depth does not remove state/mode dimensions, edge-solve time, validation or repeated attempts.

Established: bounded stored correction rank, explicit per-pass costs, and preservation of convergence when the existing original-problem safeguard and fallback are used correctly. **Empirical:** the useful rank, reference quality, acceptance rate and total time to the requested gap. A small rank guarantees cheaper tree algebra; it does not guarantee a useful correction or a small number of rounds. Compare rank choices by total checked progress per unit time, including failures and fallback work.

Selected references

[1] T. Marcucci, J. Umenberger, P. Parrilo, R. Tedrake. Shortest paths in graphs of convex sets. SIAM J. Optim., 2024.

[2] A. Chambolle, T. Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. JMIV, 2011.

[3] T. Pock, A. Chambolle. Diagonal preconditioning for first order primal-dual algorithms. ICCV, 2011.

[4] D. Applegate, O. Hinder, H. Lu, M. Lubin. Faster first-order primal-dual methods for linear programming using restarts and sharpness. Math. Prog., 2023.

[5] J. Rasch, A. Chambolle. Inexact first-order primal–dual algorithms. Comput. Optim. Appl., 2020.

[6] A. Themelis, P. Patrinos. SuperMann: a superlinearly convergent algorithm for finding fixed points of nonexpansive operators. IEEE TAC, 2019.

[7] M. Solodov, B. Svaiter. A hybrid approximate extragradient–proximal point algorithm using the enlargement of a maximal monotone operator. Set-Valued Anal., 1999.

[8] I. Nielsen, D. Axehill. A parallel Riccati factorization algorithm with applications to model predictive control. arXiv:1407.6898.

[9] O. Fercoq, A. Gramfort, J. Salmon. Mind the duality gap: safer rules for the Lasso. ICML, 2015.

[10] C. Dellin, S. Srinivasa. Lazy best-first search over paths with expensive edge evaluations (LazySP). ICAPS, 2016.

[11] A. Bemporad, M. Morari, V. Dua, E. Pistikopoulos. The explicit linear quadratic regulator for constrained systems. Automatica, 2002.

[12] A. Geoffrion. Lagrangean relaxation for integer programming. Math. Prog. Study, 1974.

[13] N. Z. Shor. Quadratic optimization problems. Soviet J. Comput. Syst. Sci., 1987.

[14] R. Featherstone. Rigid Body Dynamics Algorithms. Springer, 2008.